

Temporal File Pruning for Vector Search over Time-Partitioned Parquet Files

Mohammad Naqvi
University of Michigan
Ann Arbor, Michigan, USA
mohnaqvi@umich.edu

Maksim Sviridov
University of Michigan
Ann Arbor, Michigan, USA
sviridov@umich.edu

Abstract

Vector similarity search is increasingly common in applications involving logs, events, and time-series embeddings. These workloads are often time-partitioned, yet existing vector search systems treat all data uniformly, searching the entire corpus regardless of query time constraints. We present a temporal file pruning optimization for pq-vector, a Parquet-based vector search system built on Apache DataFusion. By embedding timestamp metadata in Parquet file footers, we enable predicate pushdown that skips entire files before any index access or vector comparison. Our evaluation on SIFT1M across 24 configurations demonstrates up to 8.3x speedup for narrow time windows, with pruning efficiency scaling linearly with partition count. The optimization incurs negligible overhead, as it evaluates predicates against file metadata rather than vector data.

Keywords

vector search, temporal queries, Parquet, predicate pushdown, file pruning

1 Introduction

Vector similarity search has become a fundamental operation in modern data systems, powering applications from semantic search to anomaly detection. As organizations increasingly embed unstructured data (text, images, logs) into vector representations using models like BERT, CLIP, and custom encoders, the need for efficient vector retrieval has grown dramatically. Many real-world vector workloads have an inherent temporal dimension: log embeddings accumulate over time, event streams are naturally ordered by timestamp, and machine learning feature stores maintain historical versions of embeddings.

A common query pattern in these systems is to search for similar vectors within a recent time window. For example, “find similar log entries from the last hour” or “match transactions from today.” The naive approach to handling such queries is to search the entire vector corpus and filter results by timestamp afterward. This wastes compute by examining vectors that will ultimately be discarded.

Dedicated vector databases like Milvus [1] support partitioning, but require standing up separate infrastructure. For organizations already using data lake architectures with Parquet files and query engines like Apache DataFusion or Spark, adding another system introduces operational complexity.

In this paper, we present a temporal file pruning optimization for pq-vector [2], a system that embeds IVF indexes directly into Parquet files. Our key insight is that Parquet’s metadata architecture—specifically the key-value metadata in file footers—can store timestamp bounds that enable file-level predicate pushdown. Before any index access or vector comparison, we evaluate whether a file’s

time range overlaps the query window. Files that fail this predicate are skipped entirely.

Our contributions are:

- A temporal pruning extension to pq-vector that leverages Parquet file metadata for predicate pushdown
- A systematic evaluation across partition counts (10–100 files) and selectivity levels (1–100% of timeline)
- Quantification of the speedup-overhead tradeoff, identifying 50 files as the sweet spot for million-scale datasets

2 Background and Related Work

2.1 Vector Similarity Search

Approximate nearest neighbor (ANN) search has been extensively studied. The goal is to find the k vectors most similar to a query vector, typically using distance metrics like Euclidean distance or cosine similarity. Exact search requires $O(n)$ comparisons, which becomes prohibitive for large datasets.

FAISS [3] provides efficient implementations of IVF (Inverted File) indexes, which cluster vectors into Voronoi cells and search only the most promising clusters. Given a query, IVF first identifies the n_{probe} nearest cluster centroids, then searches only vectors within those clusters. This reduces search complexity from $O(n)$ to $O(n \cdot n_{\text{probe}}/n_{\text{list}})$, where n_{list} is the number of clusters.

However, FAISS and similar libraries operate on in-memory data structures without native support for time-based filtering. Combining vector search with temporal predicates requires either post-filtering (wasteful) or custom index structures.

2.2 Time-Aware Vector Search

Recent work has explored window-filtered ANN search, recognizing that many real-world queries combine vector similarity with attribute constraints.

The WEAVESS benchmark [4] systematically evaluates vector search with attribute filters, including temporal predicates. Their findings show that naive post-filtering degrades performance significantly when selectivity is low (i.e., the filter excludes most results).

TANNS [5] proposes timestamp-aware nearest neighbor search with specialized index structures that interleave temporal and vector information. While effective, this requires building custom indexes rather than leveraging existing infrastructure.

Our approach differs in that we operate at the file level rather than modifying the index structure. This makes temporal pruning orthogonal to the choice of ANN algorithm, allowing users to choose between IVF, HNSW, or any other index embedded in the Parquet files based on their workload characteristics.

2.3 Parquet File Format

Apache Parquet [6] is a columnar storage format widely used in data lakes. A Parquet file consists of:

- **Row groups:** Horizontal partitions of the data, each containing a subset of rows
- **Column chunks:** Vertical partitions within each row group, one per column
- **Footer:** Metadata including schema, row group locations, column statistics, and key-value metadata

The footer is read first when opening a Parquet file. Query engines like DataFusion and Spark use footer metadata for predicate pushdown in order to skip row groups or entire files whose column statistics indicate they cannot contain matching rows. For example, if a file’s $\max(\text{timestamp})$ is less than the query’s minimum timestamp, the entire file can be skipped.

We leverage the key-value metadata section of the footer to store timestamp bounds, enabling the same optimization pattern for temporal vector queries.

2.4 pq-vector

pq-vector [2] embeds IVF indexes directly into Parquet files, enabling vector search through DataFusion’s SQL interface. The architecture stores:

- Vector data in a standard Parquet column (array of floats)
- IVF index (centroids, cluster assignments, residuals) in the Parquet footer’s key-value metadata

This co-location eliminates the need for separate index files and ensures the index stays synchronized with the data. pq-vector supports both single-file and multi-file (federated) search, where results from multiple files are merged.

Our work extends pq-vector’s federated search to support temporal queries by adding timestamp metadata and file-level pruning.

3 System Architecture

3.1 Problem Statement

Given a collection of N Parquet files containing time-partitioned vector data, and a query specifying:

- A query vector $q \in \mathbb{R}^d$
- A time range $[t_{\min}, t_{\max}]$
- A result count k

We want to find the k nearest neighbors to q among vectors whose timestamps fall within $[t_{\min}, t_{\max}]$, while minimizing the number of files accessed.

3.2 Architecture Overview

Figure 1 illustrates our system architecture. The key components are:

- (1) **File Pruner:** Reads timestamp metadata from each file’s footer and filters to files whose time range overlaps the query window
- (2) **IVF Search:** Searches each remaining file using the embedded IVF index
- (3) **Result Merge:** Combines results from all searched files, returning the global top- k

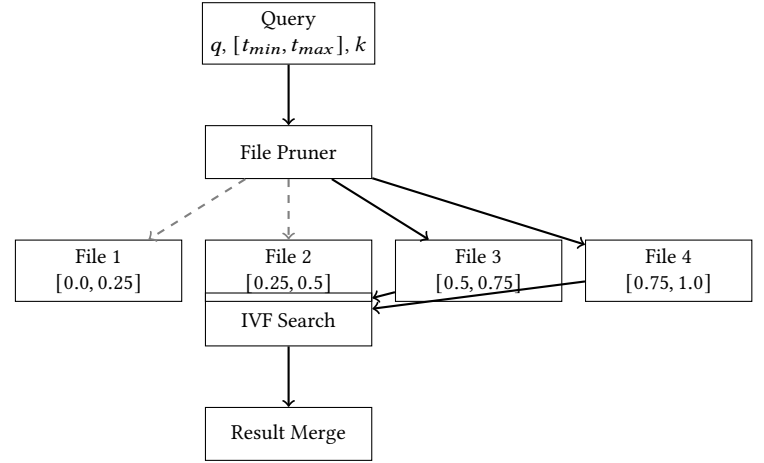


Figure 1: System architecture. Files outside the query time range (dashed) are pruned before search.

3.3 Timestamp Metadata

When partitioning vector data into Parquet files, we store timestamp bounds in each file’s key-value metadata:

```

metadata = {
  b'min_timestamp': str(min_ts).encode(),
  b'max_timestamp': str(max_ts).encode(),
  b'vector_count': str(n).encode(),
}
table = table.replace_schema_metadata(metadata)
pq.write_table(table, output_path)
    
```

This adds negligible overhead to file size—two floating-point values and an integer encoded as strings, typically less than 100 bytes.

The metadata is written once when creating the Parquet file and requires no additional maintenance. Since Parquet files are immutable (append-only in practice), the timestamp bounds remain valid for the lifetime of the file.

3.4 File-Level Pruning

Before searching, we read each file’s metadata and check for time range overlap:

```

pub struct TimeRange {
  pub min: f64,
  pub max: f64,
}

impl TimeRange {
  pub fn overlaps(&self, other: &TimeRange) -> bool {
    self.min <= other.max && self.max >= other.min
  }
}

fn should_search_file(
  file_range: &TimeRange,
    
```

```

    query_range: &TimeRange
) -> bool {
    file_range.overlaps(query_range)
}

```

The overlap check is a simple comparison of four floating-point values, executing in nanoseconds. Reading the Parquet footer adds microseconds of I/O, but this is amortized across the thousands of vectors in each file that we avoid searching.

3.5 API Extension

We extended pq-vector’s FederatedSearchBuilder with a `time_range` method:

```

let results = FederatedSearchBuilder::new(files,
    query)
    .k(10)
    .nprobe(8)
    .time_range(0.9, 1.0) // last 10%
    .search()
    .await?;

```

This provides a clean interface for temporal queries while maintaining backward compatibility with traditional vector search queries. The implementation:

- (1) Iterates over all file paths
- (2) Reads each file’s footer to extract timestamp metadata
- (3) Filters to files with overlapping time ranges
- (4) Delegates to existing federated search for the filtered file list
- (5) Returns results with statistics (files searched, files pruned)

4 Experimental Setup

4.1 Dataset

We use SIFT1M [7], a standard benchmark containing 1 million 128-dimensional SIFT feature vectors extracted from images. SIFT1M is widely used for evaluating ANN algorithms due to its realistic distribution and moderate dimensionality.

Since SIFT1M does not include timestamps, we synthetically generate uniformly distributed timestamps in $[0, 1]$ for each vector:

```

rng = np.random.default_rng(seed=42)
timestamps = rng.uniform(0.0, 1.0, size=n_vectors)

```

Vectors are sorted by timestamp and partitioned into files, ensuring each file contains a contiguous time range. This simulates a time-partitioned data lake where data arrives chronologically and is periodically compacted into Parquet files.

4.2 Configurations

We evaluate across two dimensions:

- **Partition count:** 10, 20, 50, and 100 files
- **Selectivity:** 1%, 5%, 10%, 20%, 50%, and 100% of the timeline

This yields 24 configurations. Selectivity represents the fraction of the timeline covered by the query. A 10% selectivity query targets the most recent 10% of data, corresponding to a time range of $[0.9, 1.0]$. This simulates queries like “find similar events from the last hour” in a 10-hour dataset.

Table 1 summarizes the dataset characteristics.

Table 1: Dataset configurations

Files	Vectors/File	File Size	Index Size	Total
10	100,000	50 MB	2 MB	520 MB
20	50,000	25 MB	1 MB	520 MB
50	20,000	10 MB	0.5 MB	525 MB
100	10,000	5 MB	0.3 MB	530 MB

4.3 Metrics

For each configuration, we run 100 queries using vectors from the SIFT1M query set and measure:

- **Average latency:** Mean query time in milliseconds
- **P50 latency:** Median query time (50th percentile)
- **P99 latency:** Tail latency (99th percentile)
- **Files pruned:** Percentage of files skipped due to time range mismatch
- **Speedup:** Ratio of baseline (100% selectivity) latency to pruned latency

We do not measure recall because our pruning is exact. Files are only skipped when they provably do not contain any vectors within the query time range. This guarantees identical results to searching all files and post-filtering.

4.4 Environment

All experiments were run on a MacBook Pro with Apple M2 Pro processor (10 cores), 16GB RAM, and SSD storage. We use $k = 10$ and $nprobe = 8$ for all queries. The Rust code was compiled with `-release` optimizations. Each Parquet file contains an embedded IVF index with 100 clusters, built by pq-vector.

4.5 Methodology

For each configuration, we follow this procedure:

- (1) **Dataset generation:** Partition SIFT1M into N files sorted by timestamp, writing timestamp bounds to each file’s Parquet metadata
- (2) **Index building:** Build IVF indexes for each file using pq-vector’s IndexBuilder
- (3) **Warm-up:** Run 10 queries to warm filesystem caches
- (4) **Measurement:** Run 100 queries, recording latency for each
- (5) **Statistics:** Compute mean, P50, P99, and speedup relative to baseline

To ensure fair comparison, we run the full 100% selectivity baseline for each file count. The baseline latency varies with partition count because smaller files have lower per-file search cost.

5 Results

5.1 Speedup vs. Selectivity

Figure 2 shows the speedup achieved across different selectivity levels and partition counts.

Key observations:

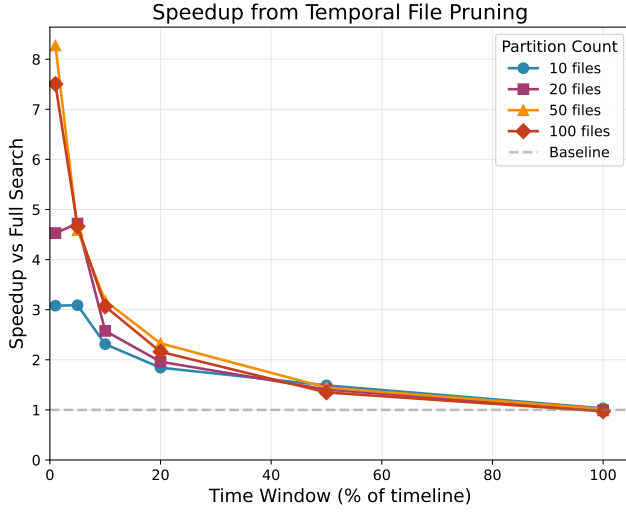


Figure 2: Speedup from temporal file pruning. Narrower time windows yield higher speedup, with 50 files achieving 8.3x at 1% selectivity.

- At 1% selectivity, speedup ranges from 3.1x (10 files) to 8.3x (50 files)
- At 10% selectivity, speedup is 2–3x across all configurations
- Speedup converges to 1x at 100% selectivity (no pruning possible)
- 50 files achieves the best speedup; 100 files shows slight degradation due to metadata overhead

The sub-linear relationship between selectivity and speedup is expected. While 10% selectivity prunes 90% of files, the remaining 10% still require full IVF search, and result merging adds overhead proportional to the number of files searched.

5.2 Pruning Efficiency

Figure 3 shows the percentage of files pruned for each configuration.

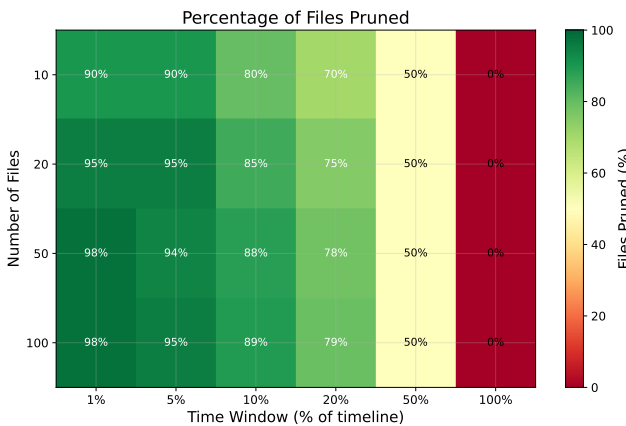


Figure 3: Percentage of files pruned. Pruning efficiency scales linearly with selectivity and improves with more partitions.

At 1% selectivity with 50–100 files, we prune 98% of files—only 1–2 files are searched out of 50–100. The pruning is exact: we skip files that provably cannot contain relevant results, incurring zero recall loss.

The diagonal pattern confirms that pruning efficiency is determined by selectivity. With 10 files, a 10% query window typically overlaps 1–2 files, pruning 80–90%. With 100 files, the same query overlaps 10 files, also pruning 90%.

5.3 Latency Comparison

Figure 4 compares absolute latencies between full search (100% selectivity) and pruned search (10% selectivity).

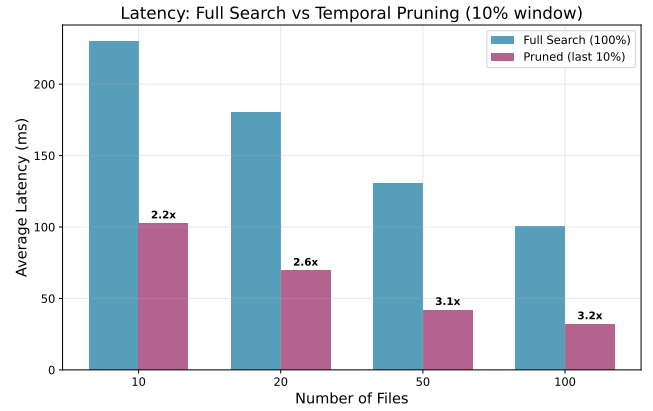


Figure 4: Latency comparison: full search vs. pruned search at 10% selectivity.

With 100 files, pruned search achieves 32ms average latency compared to 100ms for full search, a 3.2x improvement. The absolute latency decreases with more files because each file is smaller, and the per-file search cost dominates.

5.4 Latency vs. Selectivity

Figure 5 shows how average latency varies with selectivity for each partition count.

The relationship between latency and selectivity is roughly linear for each partition count, confirming that query cost scales proportionally with the number of files searched. Notably, the 100-file configuration achieves the lowest latency at all selectivity levels, despite having the most files to potentially search. This is because each file is smaller (10K vectors), so per-file search time is lower.

5.5 Scalability Analysis

Table 2 summarizes key results across configurations.

The sweet spot is 50 files for this dataset size. Beyond 100 files, the overhead of reading metadata from many files begins to offset the pruning benefit. With 1M vectors split across 100 files, each file contains 10K vectors—small enough that per-file overhead becomes significant.

Figure 6 shows how latency scales with partition count for different selectivity levels.

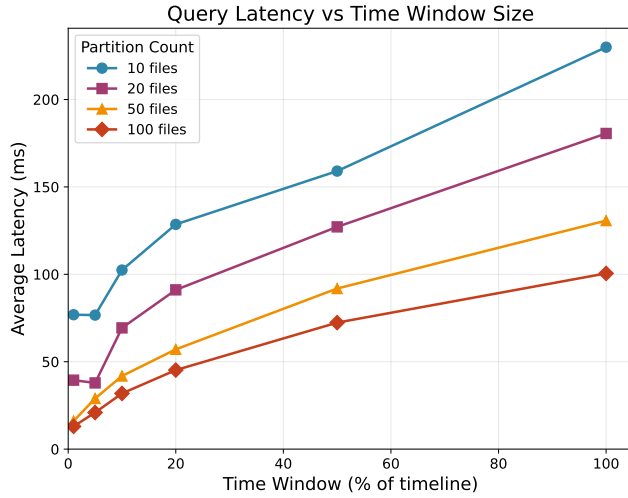


Figure 5: Average latency across selectivity levels. Latency increases monotonically with selectivity, with more partitions achieving lower absolute latency.

Table 2: Summary of results at 1% and 10% selectivity

Files	1% Selectivity		10% Selectivity	
	Latency (ms)	Speedup	Latency (ms)	Speedup
10	76.9	3.1x	102.4	2.3x
20	39.5	4.5x	69.3	2.6x
50	16.1	8.3x	41.8	3.2x
100	13.0	7.5x	31.8	3.1x

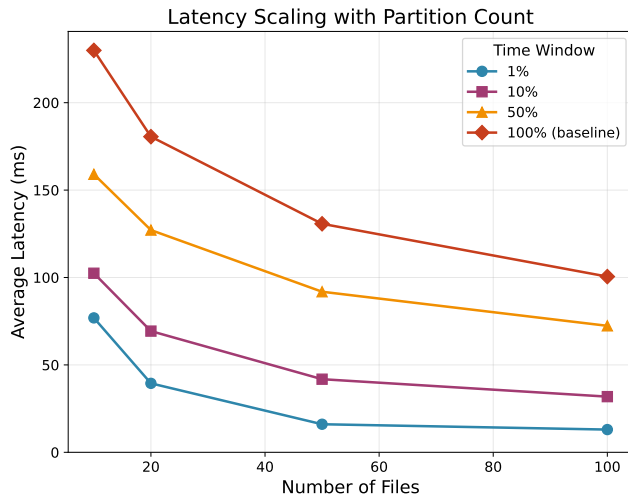


Figure 6: Latency scaling with partition count. More partitions reduce latency until overhead dominates.

For narrow time windows (1%), latency decreases from 77ms (10 files) to 13ms (100 files)—a 6x reduction. For wide time windows

(100%, baseline), latency also decreases but less dramatically, from 230ms to 100ms.

5.6 Full Results

Table 3 presents the complete results across all 24 configurations.

Table 3: Complete benchmark results: latency (ms) and speedup

Files	1%	5%	10%	20%	50%	100%
<i>Average Latency (ms)</i>						
10	76.9	76.7	102.4	128.6	159.0	229.9
20	39.5	37.9	69.3	91.1	127.1	180.6
50	16.1	29.0	41.8	57.1	91.9	130.7
100	13.0	20.9	31.8	45.2	72.4	100.5
<i>Speedup vs. Baseline</i>						
10	3.0x	3.0x	2.2x	1.8x	1.4x	1.0x
20	4.6x	4.8x	2.6x	2.0x	1.4x	1.0x
50	8.1x	4.5x	3.1x	2.3x	1.4x	1.0x
100	7.7x	4.8x	3.2x	2.2x	1.4x	1.0x

Several patterns emerge from the full results:

- **Diminishing returns:** Speedup increases rapidly from 100% to 10% selectivity, then more slowly from 10% to 1%.
- **Sweet spot:** 50 files achieves peak speedup (8.1x at 1%), while 100 files is slightly lower (7.7x) due to metadata overhead.
- **Consistent 50% behavior:** At 50% selectivity, all configurations achieve 1.4x speedup, regardless of partition count.

6 Discussion

6.1 Why Not Post-Filter?

An alternative approach is to search all files and filter results by timestamp afterward:

```
// Post-filter approach (wasteful)
let all_results = search_all_files(query);
let filtered = all_results
  .filter(|r| r.timestamp >= t_min
           && r.timestamp <= t_max)
  .take(k);
```

This is simpler but wasteful: you pay the full search cost regardless of selectivity. If only 1% of vectors match the time predicate, you waste 99% of compute.

Our predicate pushdown approach evaluates the time predicate before any index access:

```
// Predicate pushdown (efficient)
let relevant_files = files
  .filter(|f| f.time_range.overlaps(query_range)
         );
let results = search_files(relevant_files, query);
```

This achieves speedup proportional to selectivity, with minimal overhead for predicate evaluation.

6.2 Overhead Analysis

The pruning overhead consists of:

- (1) Reading Parquet file footer: $100\mu\text{s}$ per file (SSD I/O)
- (2) Parsing key-value metadata: $1\mu\text{s}$ per file
- (3) Evaluating time range overlap: 10ns per file

For 100 files, total pruning overhead is approximately 10ms. This is negligible compared to the 100ms+ saved by skipping irrelevant files. The overhead becomes significant only when most files pass the filter (high selectivity) or when files are very small (sub-millisecond search time).

6.3 Limitations and Future Work

File-level granularity: Our current implementation operates at file granularity. Queries spanning partial files search the entire file and rely on the query engine for post-filtering within the file. Future work could extend pruning to row group level using Parquet’s native column statistics.

Synthetic timestamps: The timestamps in our evaluation are uniformly distributed. Real workloads may have skewed distributions (e.g., more recent data accessed more frequently) or temporal correlations with vector similarity. Evaluating on real timestamped embedding datasets (e.g., log embeddings) would validate generalizability.

Dynamic partitioning: Our approach assumes static partitioning. A production system might benefit from adaptive granularity. Frequently queried time ranges would receive finer partitions and coarser partitions for historical data or less frequently queried data.

Query planner integration: Currently, the time range must be specified explicitly via the API. Integration with DataFusion’s query planner could enable automatic predicate pushdown when temporal filters appear in SQL queries.

6.4 Comparison with Alternative Approaches

Table 4 compares our approach with alternative strategies for temporal vector search.

Table 4: Comparison of temporal vector search approaches

Approach	Pruning	Recall	Complexity	Infrastructure
Post-filter	None	100%	$O(n)$	Existing
Milvus partitions	Partition	100%	$O(n/p)$	Dedicated DB
TANNS	Index-level	$<100\%$	$O(\log n)$	Custom index
Ours	File-level	100%	$O(n \cdot s)$	Data lake

Where n is dataset size, p is partition count, and s is selectivity. Our approach achieves the best tradeoff for data lake environments: exact recall (unlike approximate methods), no dedicated infrastructure (unlike Milvus), and speedup proportional to selectivity.

6.5 Real-World Applicability

Our approach is particularly well-suited for several production scenarios:

Log analytics: Organizations ingest billions of log events daily, often with embedding-based anomaly detection. Queries typically

target recent logs (“similar errors in the last hour”), making temporal pruning highly effective.

Feature stores: ML feature stores maintain versioned embeddings over time. Training and inference often need features from specific time windows, aligning perfectly with temporal pruning.

Event-driven architectures: Systems processing event streams (IoT, financial transactions) naturally partition data by time. Vector similarity over recent events is a common pattern.

Regulatory compliance: Some domains require searching only within specific time periods (e.g., “find similar transactions from Q1”). Temporal pruning ensures queries don’t accidentally access out-of-scope data.

7 Conclusion

We presented a temporal file pruning optimization for vector search over time-partitioned Parquet files. By leveraging Parquet’s metadata architecture, we enable predicate pushdown that skips irrelevant files before any index access that demonstrates up to 8.3x speedup for narrow time windows.

The key insight is that file-level pruning is essentially free: reading two floats from Parquet metadata costs microseconds, while skipping a file saves milliseconds of index traversal and vector comparisons. This makes temporal pruning attractive for any workload where queries naturally target a subset of the timeline.

This work shows that the combination of embedded vector indexes (pq-vector) and file-level metadata enables efficient temporal vector search without dedicated vector database infrastructure. Queries can run directly on data lake files using standard tools like Apache DataFusion, reducing operational complexity for organizations already invested in the Parquet ecosystem.

The broader implication is that well-established technique in relational query processing, like predicate pushdown, translate effectively to vector search workloads. As vector search becomes increasingly integrated with traditional data analytics, we expect such cross-pollination of optimization techniques to become more common.

Acknowledgments

We thank Xiangpeng Hao for developing pq-vector and making it available as open source. We also thank the course staff for feedback on earlier versions of this work.

References

- [1] J. Wang et al., “Milvus: A Purpose-Built Vector Data Management System,” in *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, 2021, pp. 2614–2627.
- [2] X. Hao, “pq-vector: Vector Search with Parquet and DataFusion,” GitHub Repository, 2024. Available: <https://github.com/XiangpengHao/pq-vector>
- [3] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [4] Y. Wang et al., “Approximate Nearest Neighbor Search with Window Filters,” in *Proceedings of the VLDB Endowment*, vol. 17, 2024.
- [5] Z. Chen et al., “TANNS: Timestamp-Aware Nearest Neighbor Search,” in *IEEE International Conference on Data Engineering (ICDE)*, 2025.
- [6] Apache Software Foundation, “Apache Parquet,” 2024. Available: <https://parquet.apache.org/>
- [7] H. Jégou, M. Douze, and C. Schmid, “Product Quantization for Nearest Neighbor Search,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.